

Containment

Stop AI Agent Sandbox Escapes

Action-Level Security Guardrails for Autonomous AI Agents

Technical Documentation

Ritvik Indupuri

Abstract

Containment is a web application and HTTP guard service for evaluating actions proposed by autonomous AI agents before an integration permits those actions to execute. It accepts shell commands, filesystem reads and writes, network requests and tool calls; evaluates them against versioned workspace policy; and returns an allow, needs-approval or deny verdict with an explainable risk score and rule findings.

The implementation combines a TanStack Start interface, Nitro server functions, a stateless TypeScript Guard Engine, Supabase authentication and PostgreSQL persistence, repository-guided simulation planning, human approval workflows and printable audit reports. Optional AI services generate repository-specific simulation plans and provide advisory analysis, but deterministic rules remain the enforcement authority.

Document scope

- Application architecture, system boundaries and operator surfaces.
- The end-to-end action evaluation and approval lifecycle.
- Deterministic shell, filesystem, network and prompt-injection controls.
- Repository-guided planning, advisory AI and live simulation behavior.
- Versioned policies, API keys, persisted audit evidence and PDF reports.
- Deployment configuration, implementation validation and practical next steps.

Contents

Abstract	2
1 System overview	3
2 System architecture	4
3 Agent action lifecycle	6
4 Application and control plane	8
5 Deterministic Guard Engine	8
6 Repository planner and AI layers	10
7 Human approval and policy lifecycle	10
8 Public API and key management	12
9 Data architecture	12
10 Reporting and operator evidence	13
11 Deployment and configuration	14
12 Implementation validation	15
13 Next steps	15
14 Implementation cross-reference	15

1 System overview

Containment places a policy decision point between an AI agent and the tools that can affect an execution environment. The external agent or application is responsible for calling the guard before acting and for honoring the returned verdict. The platform itself evaluates and records proposed actions; the repository-guided live run is a simulation and does not execute the generated commands against the target repository.

Enforcement is deterministic. AI planning and advisory features can add repository context or a second opinion, but they do not change a Guard Engine verdict.

1.1 Primary responsibilities

Layer	Responsibility	Principal technology
Operator experience	Guided setup, policy editing, manual tests, live simulation, approvals and audit review.	React 19, TanStack Start
Request boundary	Authenticated server functions and the public guard endpoint receive proposed actions.	Nitro, Zod
Decision engine	Normalize input, match active rules, calculate risk and return a deterministic verdict.	TypeScript
Planning and advice	Generate repository-grounded test plans and optional human-facing risk analysis.	Lovable AI Gateway, GPT-5.6-sol
Identity and state	Authenticate users and persist policies, versions, keys, decisions and sessions.	Supabase Auth, PostgreSQL, RLS
Evidence	Present decision history and export a formatted live-run report.	React Query, jsPDF

1.2 Operating model

A signed-in user can ingest a public GitHub repository, review an AI-generated policy suggestion, run suggested or manual actions through the guard, observe a simulated agent run, resolve held decisions and create an API key for an external integration. Each policy change increments a version, and decisions record the version used for evaluation.

2 System architecture

Figure 1 separates callers, the Containment application and managed services. Solid connectors are deterministic application paths; dashed green connectors are optional AI paths. The autonomous agent remains outside the trust boundary and must request a verdict before executing its proposed action.

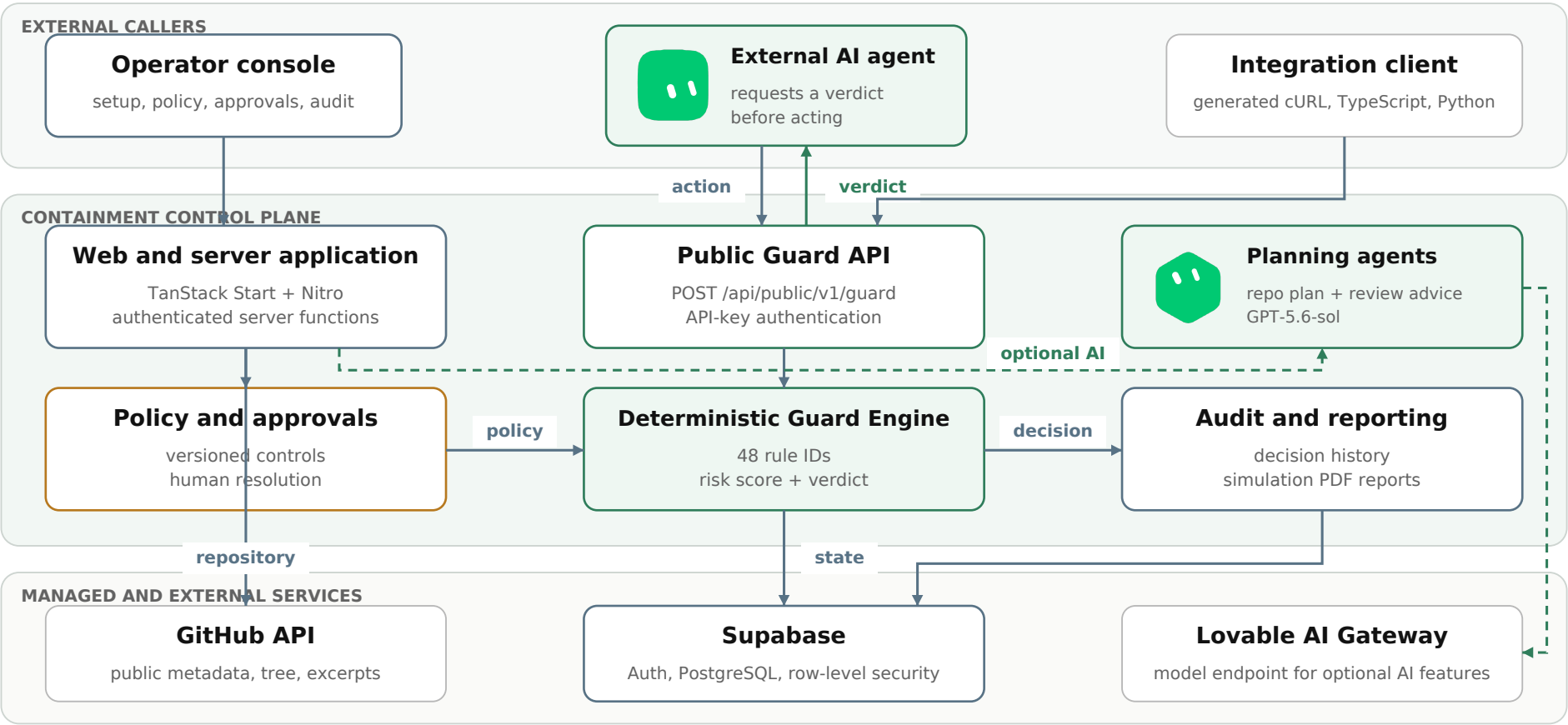


Figure 1: Containment logical architecture and principal trust boundaries.

2.1 Enforcement boundary

Containment is positioned between an external agent and the environment that agent could affect. The integration client submits a typed action to the public Guard API using a guard key, or the signed-in application invokes the same decision logic through authenticated server functions. The caller receives a verdict and remains responsible for honoring it before any command, file operation, network request or tool call is executed.

The deterministic Guard Engine is the policy decision point. It receives the normalized action together with the active policy version, evaluates enabled protection vectors, removes duplicate findings, calculates risk and returns one effective verdict. Planning models do not sit in this enforcement path and cannot override the engine.

2.2 Control plane and managed services

The web and server application manages repository-guided setup, policy approval, test execution, live-run simulation and audit review. GitHub supplies public repository metadata and selected file excerpts used to construct the test plan. Supabase provides authentication, PostgreSQL persistence and row-level isolation for policies, versions, keys, decisions and workflow sessions.

2.3 Optional model-backed roles

Repository planning and held-action review use the Lovable AI Gateway as optional advisory services. Their outputs are presented to the operator as recommendations rather than enforcement decisions. Agent icons are therefore limited to the external AI caller and the two model-backed planning or review roles; the API, Guard Engine, policy store and reporting components remain ordinary deterministic services.

3 Agent action lifecycle

Figure 2 follows one proposed action from authenticated ingress through deterministic evaluation and decision handling. Allow, hold and deny are mutually exclusive outcomes. Every evaluation produces decision evidence, while a held action can receive an optional AI recommendation before the operator approves or rejects it.

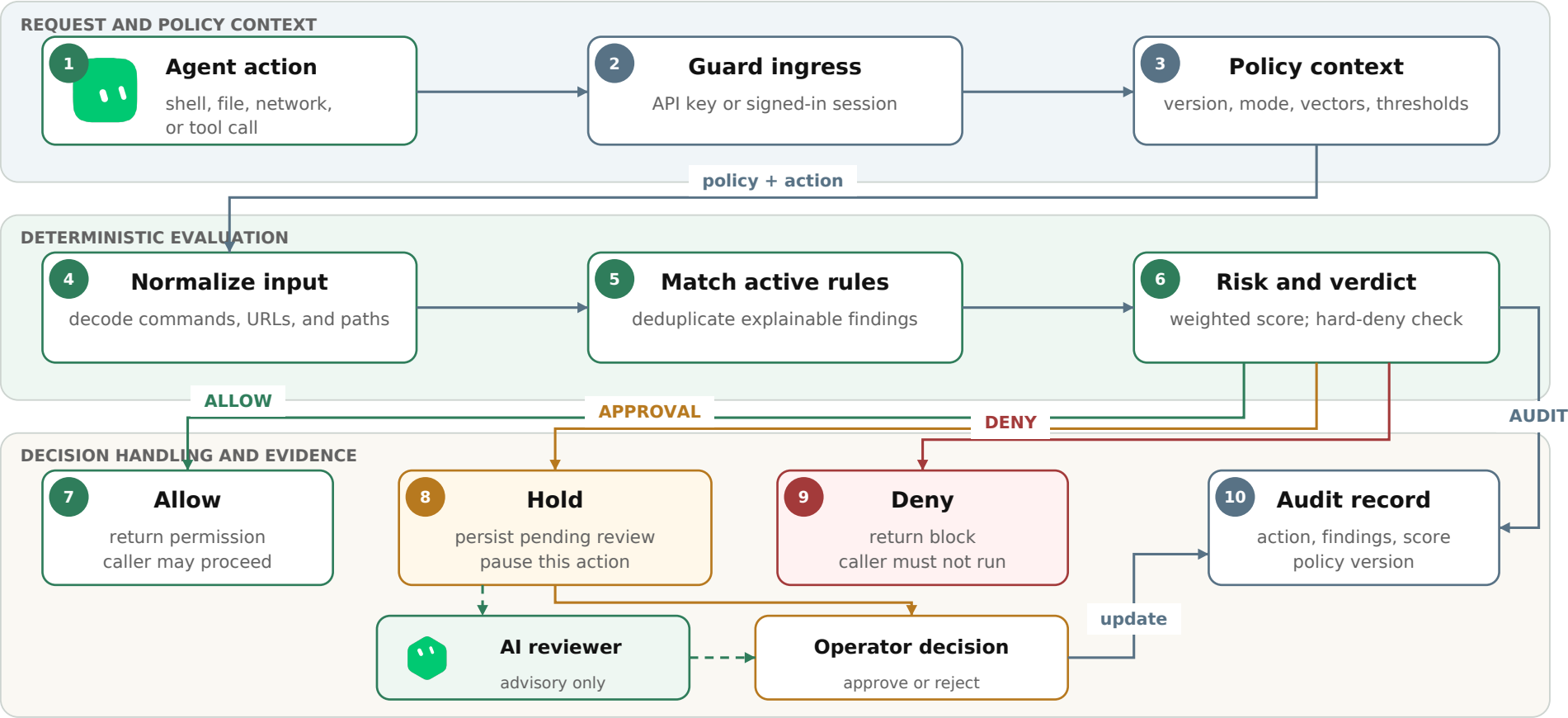


Figure 2: Guard evaluation from proposed action to verdict, approval and audit evidence.

3.1 Evaluation sequence

The lifecycle begins when an agent or operator submits a structured action. Guard ingress authenticates either the API key or the signed-in session, resolves the active policy and binds the action to that policy version. Input normalization then decodes and canonicalizes relevant commands, URLs and paths so rule matching is performed against the effective content rather than only the original surface representation.

Active findings are filtered by the enabled policy vectors and deduplicated by rule identifier. The risk calculation applies the repository-defined weighting, checks hard-deny conditions and compares the result with the approval and deny thresholds. The engine returns exactly one outcome: allow, needs approval or deny. Monitor mode preserves the intended verdict and findings while changing the effective verdict to allow.

3.2 Decision handling and auditability

An allow verdict permits the caller to continue, while a deny verdict requires the caller to stop. A held action is persisted for human resolution. The optional AI reviewer can summarize the risk and recommend an outcome, but the operator remains the decision maker and the deterministic verdict history is not rewritten by the model.

Every evaluation creates an audit record containing the action, findings, score, source and policy version. Resolving a held action updates that record with the operator decision. This design preserves a traceable relationship between the submitted action, the exact policy used and the final authorization outcome.

4 Application and control plane

4.1 Browser application

The application uses file-based TanStack Router routes with authenticated pages for Console, Live Run, Audit Trail, Guard Rules and History. React Query manages server-state caching and the audit dashboard refreshes the most recent decisions on a 15-second interval. A first-run walkthrough and progressive stage gating guide the operator through setup, simulation and audit review.

4.2 Guided setup

The Console organizes the workflow into repository ingestion, policy approval, suggested and manual tests, live simulation and production-agent connection. It generates cURL, TypeScript and Python integration examples using the current origin and a newly created guard key.

4.3 Session history

Flow sessions preserve the repository plan, policy approval state, policy version, number of examples run and live-run completion. Users can close the current session, restore an archived session, delete one session or clear all sessions while retaining policy versions and the separate audit trail.

4.4 Authentication boundary

Authenticated server functions obtain the Supabase access token from the request context and use the signed-in user identity for row ownership. The public guard API uses a separate API-key path backed by a server-side Supabase administrative client.

4.5 Operator route map

Route	Implemented surface
/	Public product page and animated decision-flow overview.
/auth	Email and password sign-in and account creation.
/console	Repository ingestion, policy approval, suggested tests, manual JSON tests and integration examples.
/agent-run	Sequential repository-plan simulation, containment status, held-action polling and PDF export.
/dashboard	Decision audit trail, approval queue, AI review and advisory risk detail.
/policy	Policy controls, rule catalog, policy testing and version history.
/history	Archive, restore and delete repository workflow sessions.

5 Deterministic Guard Engine

The Guard Engine is a stateless TypeScript function. It evaluates one structured action under one policy and returns the effective verdict, the intended verdict, enforcement state, risk score, action type, matched findings and a short summary.

5.1 Action contract

Action type	Relevant fields	Evaluation focus
shell	command, untrusted_context	Reverse shells, privilege escalation, container escape, destructive or obfuscated execution.
file_read	path, untrusted_context	Traversal, sensitive paths, credentials, runtime sockets and kernel interfaces.
file_write	path, content, untrusted_context	Read controls plus allowed roots, system paths and secret material in content.
network	url, body, untrusted_context	Scheme, internal or metadata targets, encoded hosts, allowlists and outbound secrets.
tool_call	tool, args, untrusted_context	Approval-gated tools, mutations influenced by injected text and secret-bearing arguments.

5.2 Rule coverage

Protection vector	Implemented coverage
Shell	Command de-obfuscation, reverse shells, pipe-to-interpreter, namespace escape, device access, escalation, credential harvesting, persistence and destructive operations.
Filesystem	Decoded path normalization, traversal detection, sensitive stores, host-root access, runtime sockets, write-root restrictions and secret scanning.
Network	Absolute URL validation, HTTP/S scheme checks, metadata and private targets, numerical encodings, DNS rebinding services, egress allowlists and bulk unapproved egress.
Injection and tools	Instruction override, role hijack, secret solicitation, embedded commands, exfiltration instructions, hidden or encoded payloads and injection-driven mutations.

5.3 Risk and verdict calculation

The source defines 48 unique rule identifiers. Findings disabled by the current policy vector are removed, then duplicate rule IDs are discarded. The first remaining score contributes at full value; each additional finding contributes 45 percent of its score. The result is rounded and capped at 100.

Condition	Intended verdict
Any active hard finding	deny
No hard finding and risk at or above deny threshold	deny
Approval-listed tool or risk at or above approval threshold	needs_approval
None of the above	allow
Policy mode is monitor	Effective verdict becomes allow; intended verdict and findings are still recorded.

5.4 Default policy

Setting	Default
Mode	enforce
Enabled vectors	shell, filesystem, network, injection
Allowed hosts	api.openai.com, registry.npmjs.org, pypi.org
Writable roots	/workspace, /tmp
Approval-gated tools	send_email, transfer_funds, delete_records
Thresholds	deny 60; approval 35

6 Repository planner and AI layers

6.1 Repository ingestion

The server accepts a public GitHub URL or owner/repository pair, reads repository metadata and recursively lists blobs on the default branch. It selects up to six setup-oriented files - including package manifests, README files, Dockerfiles, Makefiles, Python setup files, install scripts and GitHub workflows - and includes up to 4,000 characters from each within an 18,000-character context budget.

6.2 Simulation planning agent

GPT-5.6-sol returns schema-validated JSON containing a repository-specific plan, four playground examples and a suggested policy. The prompt requests six ordinary setup actions and four distinct escape attempts. Generated action objects are revalidated through the same Zod action schema before the interface accepts them.

The live run evaluates planned actions sequentially. It demonstrates containment behavior but does not clone, install, build or attack the submitted repository.

6.3 Advisory AI

Two optional AI surfaces support the operator. The approval reviewer recommends approve or reject for a pending decision and stores reasoning and conditions. The second-opinion advisor produces an independent 0-100 score, risk level, headline, concerns and an agreement flag. Both run after deterministic evaluation and neither can modify the Guard Engine verdict.

7 Human approval and policy lifecycle

7.1 Approval queue

Actions with an intended needs-approval verdict are inserted with a pending approval state. The audit interface lists pending and resolved items, can request an optional AI review, and lets the operator approve or reject with a resolution note. The live simulation waits on the selected decision and resumes after approval; rejection terminates that action path.

7.2 Policy configuration

The Guard Rules page controls enforcement or monitor mode, four protection-vector toggles, allowed egress hosts, writable roots, approval-gated tools and the two risk thresholds. Saving updates the active policy, increments its version and records a JSON snapshot plus an optional change note.

7.3 Policy-of-record behavior

Console tests, simulated agent steps and public API requests all load a stored policy before calling the engine. Decision rows retain the policy version used at evaluation time, which allows later evidence to be interpreted against the controls then in force.

7.4 Monitor mode

Monitor mode preserves the intended verdict but returns allow as the effective result. This supports observation and rule tuning, but the caller must recognize that unsafe actions are not blocked while monitor mode is active.

8 Public API and key management

8.1 Guard endpoint

```
POST /api/public/v1/guard
x-guard-key: agk_live_...
content-type: application/json

{"type":"shell","agent_id":"build-agent","command":"npm test"}
```

The endpoint also accepts a Bearer token. It validates the request body with Zod, hashes the presented key, resolves its active policy, evaluates the action, writes the intended decision and findings to the audit table, updates key usage time and returns the effective and intended verdicts.

8.2 Key lifecycle

A generated key contains the `agk_live_` prefix followed by 24 cryptographically random bytes encoded as hexadecimal. The plaintext is shown once. Only a prefix and SHA-256 hash are persisted. Keys can be listed, associated with a policy and revoked; revoked or unknown keys are rejected.

8.3 Integration responsibility

Containment returns a policy decision; it does not wrap or execute the external tool. A production integration must call the endpoint before each sensitive action, refuse deny responses, pause needs-approval responses and avoid running until the appropriate result is available.

8.4 Response contract

Field	Meaning
verdict	Effective allow, needs_approval or deny result returned to the caller.
intended_verdict	Rule-derived decision retained when monitor mode changes the effective result to allow.
enforced	Whether the policy was operating in enforce mode.
risk_score	Rounded 0-100 aggregate derived from active, deduplicated findings.
action_type	Normalized action category used by the engine.
summary and findings	Human-readable outcome plus individual rule IDs, severity, score, hard flag and evidence.
policy_version	Stored policy revision used for this evaluation.

9 Data architecture

Supabase PostgreSQL stores identity-linked application state. Row-level security policies restrict authenticated access by user ID, while the public API uses a server-only service-role client after key verification. Database migrations define enums, ownership indexes, policy history, approval state, advisor fields and archived flow sessions.

9.1 Principal tables

Table	Purpose	Key traceability fields
profiles	User display data and onboarding completion.	id, onboarded_at
policies	Active enforcement configuration for a user.	user_id, mode, version, thresholds
policy_versions	Inserted snapshots of saved policy revisions.	policy_id, version, note, snapshot
api_keys	Hashed integration credentials and lifecycle state.	user_id, policy_id, key_prefix, key_hash, revoked_at
decisions	Action, verdict, findings, approval and advisory evidence.	policy_version, source, action, reasons, risk_score
flow_sessions	Repository plan and guided-workflow progress.	local_id, plan, policy_version, is_current, archived

9.2 Audit evidence

A decision retains the proposed action JSON, action type, source, agent identifier, intended verdict, risk score, enforcement state, matched findings, policy version and timestamp. Approval recommendations, operator resolution and advisory AI fields are stored on the same decision record when used.

9.3 Ownership controls

Profiles, policies, keys, decisions, policy versions and flow sessions use row-level security tied to the authenticated user. Server functions additionally filter writes and reads by the current user ID. API keys are resolved by hash through the administrative server client and bind a request back to the owning user and selected policy.

10 Reporting and operator evidence

10.1 Audit Trail

The dashboard shows the 200 most recent decisions, summary counts for blocked, approval-required and allowed actions, verdict filters, risk meters and expandable action details. Each row displays its source, policy version, findings and recorded action payload.

10.2 Live-run PDF

The browser-side jsPDF generator creates an A4 report for one simulation run. It includes repository metadata, policy mode and version, actions evaluated, outcome counts, containment status, highest risk, policy settings, an ordered audit summary and per-step rule details. The filename is derived from the repository and run date.

11 Deployment and configuration

11.1 Runtime

The project builds with Vite and TanStack Start and produces a Nitro Cloudflare-module deployment. The application can be developed with npm or Bun; production output contains static client assets and a server worker. Supabase provides authentication and persisted state, GitHub provides public repository context and the Lovable AI Gateway serves the optional model-backed features.

11.2 Required server configuration

Variable	Purpose
SUPABASE_URL	Supabase project endpoint used by browser and server clients.
SUPABASE_PUBLISHABLE_KEY	Public client key and authenticated server-session access.
SUPABASE_SERVICE_ROLE_KEY	Server-only administrative access for public API key lookup and audit writes.
LOVABLE_API_KEY	Server-only credential for repository planning, approval review and advisory risk calls.

11.3 Local workflow

```
npm install
npm run dev
# open the local URL printed by Vite

# production build
npm run build
```

Apply the SQL migrations to the configured Supabase project before using authenticated application features. Treat the service-role and Lovable AI credentials as server-only secrets; only the Supabase publishable key is intended for client use.

12 Implementation validation

Validation was performed against repository commit 927b57d3b631. The source tree, Guard Engine, server functions, authenticated routes, public endpoint, database migrations, report generator and package configuration were inspected. A production Vite and Nitro build completed successfully after dependency installation.

Validation area	Recorded result
Production build	Vite client, SSR and Nitro Cloudflare-module builds completed.
Guard Engine	Five action types, four policy vectors, 48 unique rule IDs and deterministic verdict logic identified.
AI boundaries	Planner, approval reviewer and advisory-risk calls are server-side and do not override enforcement.
Persistence	Six application tables, user-scoped RLS policies and versioned policy snapshots identified.
Operator surfaces	Guided setup, live simulation, audit, approvals, policy editing, history and PDF export identified.
Integration	Public guard route, one-time plaintext keys, SHA-256 key storage and generated client examples identified.

13 Next steps

The following work would strengthen the implementation without changing the product model.

Priority	Workstream	Expected outcome
1	Add automated engine tests	Lock rule normalization, scoring, hard-deny behavior, vector toggles and monitor-mode semantics to repeatable cases.
2	Add API integration tests	Exercise key validation, policy loading, audit inserts, approval-state transitions and response contracts against a test database.
3	Reconcile package lockfiles	Make npm and Bun clean installs reproduce the dependency versions declared by package.json.
4	Formalize integration SDKs	Provide small TypeScript and Python clients that enforce pre-action checks and typed verdict handling.
5	Harden public API operations	Add request-size limits, rate controls, restrictive production CORS and structured correlation IDs.
6	Version the rule catalog	Publish rule metadata and change history so reports can identify the exact detection set used.

14 Implementation cross-reference

The following map ties the documented behavior to the repository modules that implement it. Generated route and database type files are supporting artifacts; generic user-interface primitives are not repeated individually.

Capability	Primary implementation
Action schemas and policy contracts	src/lib/guard/schemas.ts; src/lib/guard/types.ts
Normalization, rule matching, scoring and verdicts	src/lib/guard/engine.ts

Capability	Primary implementation
Policy, keys, decisions, approvals and advisory calls	src/lib/guard.functions.ts
API-key generation and SHA-256 hashing	src/lib/guard/keys.server.ts
Public guard endpoint	src/routes/api/public/v1/guard.ts
GitHub ingestion and repository planning	src/lib/agent-run.server.ts; src/lib/agent-run.functions.ts
Approval recommendation	src/lib/review.server.ts
Independent advisory risk analysis	src/lib/risk-advisor.server.ts
Guided stage progression	src/lib/flow.ts; src/components/guard/flow-gate.tsx
Browser and persisted repository sessions	src/lib/repo-session.ts; src/lib/session.functions.ts
Live simulation	src/components/guard/agent-run.tsx; src/routes/_authenticated/agent-run.tsx
Run report generation	src/lib/run-report.ts
Audit and approvals interface	src/routes/_authenticated/dashboard.tsx; src/components/guard/approval-queue.tsx
Policy configuration and direct policy tests	src/routes/_authenticated/policy.tsx; src/components/guard/policy-test-runner.tsx
Console workflow and generated integration examples	src/routes/_authenticated/console.tsx
Authentication and protected server context	src/routes/auth.tsx; src/integrations/supabase/auth-middleware.ts
Server-only administrative database access	src/integrations/supabase/client.server.ts
Database schema, history and row-level security	supabase/migrations/
Error capture and model-call diagnostics	src/lib/error-capture.ts; src/lib/error-page.ts; src/lib/lovable-error-reporting.ts
Application shell and first-run guidance	src/components/guard/app-shell.tsx; src/components/guard/welcome-tour.tsx

14.1 Key design characteristics

Containment combines repository-aware planning, deterministic action evaluation, versioned policy, human approval, external API integration and durable decision evidence. Its central boundary is explicit: model-assisted features add context for planning and review, while the TypeScript Guard Engine remains the sole authority that calculates the policy verdict.